Optimized Technology Stack

This architecture has been streamlined to rely on native .NET components and React-based SDKs, eliminating unnecessary external dependencies while fulfilling all core project features.

Subsystem	Selected Technology	Project Features Fulfilled
Frontend UI	React.js	Rapid component development and state management.
Video UI / SDK	`@livekit/components-react`	Audio (5), Video (6), Screen Sharing (8), Device selection.
Backend Core	.NET Core Web API	Business logic, robust routing, and meeting room generation.
Authentication	ASP.NET Core Identity	Account & Identity (1), User Management (2), Admin Portal (25).
Real-time State	.NET SignalR	Meeting Chat (9), Participant Controls (10), Waiting Room (11).
Database	MSSQL Server (Docker)	Meeting Scheduling (13), History (20/30), unified user schema.
Media SFU	LiveKit Server	WebRTC & Signaling layer (7) – fully automated media routing.
Whiteboard	`tldraw` SDK	Whiteboard (22) – Synced via LiveKit data channels.
Recording	LiveKit Egress	Recording (17), Server-side composite rendering.
Object Storage	MinIO (Docker)	File / Recording Storage (18/28) via S3 compatibility.

> **Crucial Shift: Bypassing Manual Signaling**
>
> The manual requirement for "ICE --> SDP --> Media negotiation" (Stages 6 & 7) is entirely absorbed by LiveKit. Using the React SDK completely eliminates the need to build a fragile, custom WebRTC signaling broker over SignalR, drastically reducing project failure risk.

Workflow Implementations (Stages 1–5)

The project's required stages map directly to this simplified architecture workflow.

Stage 1: Identity & Access

React posts credentials to the .NET API. ASP.NET Core Identity hashes the password, validates it against MSSQL, and issues a self-signed JWT. No external Keycloak container is required.

Stage 2: Meeting Lifecycle

When a host creates a meeting, .NET stores the Meeting ID in MSSQL. To join, .NET validates the user's JWT and uses the LiveKit SDK to mint a secure video access token.

Stage 3: Roster Management

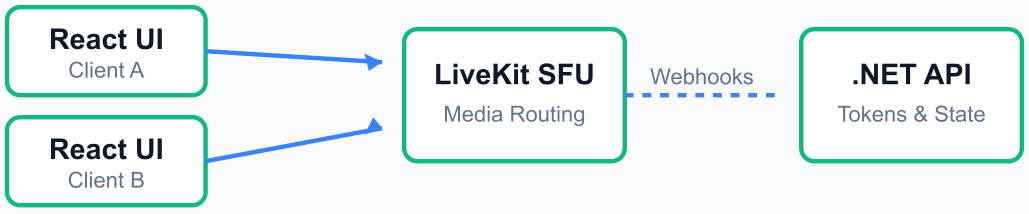
Participant lists (join/leave) are synchronized natively. When a user connects to the room, LiveKit fires a webhook back to the .NET API to log the activity in the Meeting History table.

Stages 4 & 5: Media Controls

Camera, microphone, and device selections are handled exclusively on the frontend. React's `<TrackToggle>` and `<MediaDeviceSelect>` components manage hardware permissions effortlessly.

WebRTC Connection Architecture

Unlike standard peer-to-peer applications, this setup routes all heavy traffic through the central Selective Forwarding Unit (SFU) to preserve client bandwidth and battery.



Military Intranet Deployment (Docker Swarm)

The application is packaged for strict offline (air-gapped) environments. Docker Swarm provides multi-server load balancing without the overhead of maintaining a Kubernetes cluster.

> **Offline Registry Constraints & MSSQL Integration:**
> Because the military servers lack internet access, standard `docker pull` operations will fail. Furthermore, the MSSQL Docker container requires the host machine to accept the EULA explicitly in the compose file. All dependencies must be packaged entirely on the development machine prior to physical transfer.

Execution Sequence

1. **Image Packaging:** On the internet-connected developer machine, compile the .NET API and React UI into local images. Use `docker save` to bundle these images, alongside the official `mssql/server`, LiveKit, Redis, and MinIO images into a single `zoom_project.tar` archive.
2. **Physical Transfer:** Move the `.tar` file and the `docker-compose.yml` file to the intranet servers via secure USB.
3. **Local Cache Loading:** Execute `docker load -i zoom_project.tar` on all participating servers (the Master Node and all Worker Nodes) to populate their local Docker daemon cache.
4. **Swarm Initialization:** Execute `docker swarm init` on the Master server. Copy the generated join token and execute it on the subsequent Worker nodes.
5. **Cluster Deployment:** From the Master server, execute `docker stack deploy -c docker-compose.yml zoom_app`. Swarm will automatically distribute the database, caching, video routing, and API layers across the offline hardware.